Team_21_Week3_Modelling (2)

April 19, 2026

1 Week 3 Deliverable - Modelling

Team Name: Team MAI
Project: Student Performance Prediction (Regression)
Target Variable: `G3` (Final Grade)

1.1 1. Data Linkage & Preprocessing

Recreating the datasets from Week 2 to ensure variables are defined for the models.

[9]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import cross_val_score
import warnings
warnings.filterwarnings('ignore')

# Load and Preprocess
df = pd.read_csv("student-por.csv", sep=";")
df_final = pd.get_dummies(df, drop_first=True)
X = df_final.drop('G3', axis=1)
y = df_final['G3']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

scaler = StandardScaler()
X_train = pd.DataFrame(scaler.fit_transform(X_train), columns=X.columns)
X_test = pd.DataFrame(scaler.transform(X_test), columns=X.columns)
```

```
# Results storage for side-by-side comparison
results_list = []
```

1.2 1.1 Evaluation Setup

Before evaluating the regression models, we define a unified evaluation function.

This function computes the required regression metrics on the test set: - RMSE, - MAE, - R^2.

In addition, it stores the results in a shared list so that all model performances can later be presented in a single side-by-side comparison table, as required by the project instructions.

[2]:
```
# Results storage for side-by-side comparison
results_list = []

# Unified evaluation function for regression models
def evaluate_model(model, model_name, X_test, y_test):
    y_pred = model.predict(X_test)

    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    print(f"--- {model_name} Performance ---")
    print(f"RMSE: {rmse:.4f}")
    print(f"MAE: {mae:.4f}")
    print(f"R2: {r2:.4f}")

    results_list.append({
        "Model": model_name,
        "RMSE": rmse,
        "MAE": mae,
        "R2": r2
    })
```

1.3 2. Baseline Model: Linear Regression

Per the project requirements, our first model is **Linear Regression**. This serves as our baseline benchmark. Given the high correlation between `G1`, `G2`, and `G3` discovered during EDA, we expect this model to set a high standard for performance.

[8]:
```
# 2.1 Initialize and Train
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# 2.2 5-Fold Cross-Validation
```

```
# We use R2 to check the stability of the model across different folds
lr_cv = cross_val_score(lr_model, X_train, y_train, cv=5, scoring='r2')
print(f"Baseline CV R2 Mean: {lr_cv.mean():.4f} (+/- {lr_cv.std():.4f})")

# 2.3 Evaluate on Test Set
evaluate_model(lr_model, "Linear Regression (Baseline)", X_test, y_test)
```

```
Baseline CV R2 Mean: 0.8356 (+/- 0.0433)
--- Linear Regression (Baseline) Performance ---
RMSE: 1.2149
MAE: 0.7651
R2: 0.8487
```

1.4 3. Model 2: K-Nearest Neighbors (KNN)

We chose **KNN** as our second model. Since we applied `StandardScaler` in Week 2, the distance-based calculations of KNN will be meaningful and not biased by the scale of features like `absences`.

[10]:
```
# 3.1 Initialize and Train (Using default k=5)
knn_model = KNeighborsRegressor(n_neighbors=5)
knn_model.fit(X_train, y_train)

# 3.2 5-Fold Cross-Validation
knn_cv = cross_val_score(knn_model, X_train, y_train, cv=5, scoring='r2')
print(f"KNN CV R2 Mean: {knn_cv.mean():.4f} (+/- {knn_cv.std():.4f})")

# 3.3 Evaluate on Test Set
evaluate_model(knn_model, "KNN Regressor", X_test, y_test)
```

```
KNN CV R2 Mean: 0.4901 (+/- 0.0541)
--- KNN Regressor Performance ---
RMSE: 2.1556
MAE: 1.5569
R2: 0.5235
```

1.5 4. Model 3: Decision Tree Regressor

The **Decision Tree** is implemented to capture potential non-linear interactions between variables. We have set a `max_depth` to ensure the model generalizes well to the test data and does not overfit.

[5]:
```
# 4.1 Initialize and Train
dt_model = DecisionTreeRegressor(max_depth=5, random_state=42)
dt_model.fit(X_train, y_train)

# 4.2 5-Fold Cross-Validation
```

```
dt_cv = cross_val_score(dt_model, X_train, y_train, cv=5, scoring='r2')
print(f"Decision Tree CV R2 Mean: {dt_cv.mean():.4f} (+/- {dt_cv.std():.4f})")

# 4.3 Evaluate on Test Set
evaluate_model(dt_model, "Decision Tree", X_test, y_test)
```

```
Decision Tree CV R2 Mean: 0.7618 (+/- 0.0536)
--- Decision Tree Performance ---
RMSE: 1.3355
MAE: 0.8011
R2: 0.8171
```

1.6 5. Evaluation Metrics

To compare the models fairly, we summarize the required regression metrics in a single comparison table.

For this regression task, the required metrics are: - RMSE (Root Mean Squared Error), - MAE (Mean Absolute Error), - R^2 (coefficient of determination).

These metrics allow us to compare prediction accuracy and model fit side by side across all three models.

[6]:
```
# Create comparison table from test-set metrics
results_df = pd.DataFrame(results_list)
results_df = results_df[["Model", "RMSE", "MAE", "R2"]]
results_df = results_df.round(4)

print("=== Test Set Metrics Comparison ===")
results_df
```

```
=== Test Set Metrics Comparison ===
```

[6]:

	Model	RMSE	MAE	R2
0	Linear Regression (Baseline)	1.2149	0.7651	0.8487
1	KNN Regressor	2.1556	1.5569	0.5235
2	Decision Tree	1.3355	0.8011	0.8171

1.7 6. Cross-Validation

To evaluate the stability and generalization of each model, we also report 5-fold cross-validation results.

The mean cross-validation score shows the average model performance across folds, while the standard deviation indicates how stable the model is from one fold to another.

These values are added to the same unified comparison table so that both test-set metrics and cross-validation results can be compared together.

[7]:
```
# Add cross-validation summary to the same comparison table
cv_summary = pd.DataFrame({
    "Model": [
        "Linear Regression (Baseline)",
        "KNN Regressor",
        "Decision Tree"
    ],
    "CV Mean (R2)": [
        lr_cv.mean(),
        knn_cv.mean(),
        dt_cv.mean()
    ],
    "CV Std (R2)": [
        lr_cv.std(),
        knn_cv.std(),
        dt_cv.std()
    ]
})

results_df = results_df.merge(cv_summary, on="Model")
results_df = results_df.round(4)

print("=== Final Model Comparison Table ===")
results_df
```

```
=== Final Model Comparison Table ===
```

[7]:

	Model	RMSE	MAE	R2	CV Mean (R2)	CV Std (R2)
0	Linear Regression (Baseline)	1.2149	0.7651	0.8487	0.8356	0.0433
1	KNN Regressor	2.1556	1.5569	0.5235	0.4901	0.0541
2	Decision Tree	1.3355	0.8011	0.8171	0.7618	0.0536

1.8 7.Written Interpretation

1.8.1 1. Best Performing Model

Based on the experimental results, the **Linear Regression (Baseline)** model is the top performer. It achieved the highest **R^2 score (0.8487)** and the lowest error rates (**RMSE: 1.2149, MAE: 0.7651**) on the test set. Additionally, its Cross-Validation mean R^2 of **0.8356** confirms that the

model is stable and generalizes well to unseen data, outperforming both non-linear alternatives.

1.8.2 2. Why it Outperformed Others

The superior performance of Linear Regression on this dataset suggests several things: * **Linearity of Academic Features:** The relationship between predictors (like previous grades G1/G2 and study time) and the final grade G3 appears to be predominantly linear. A simple linear boundary captures this trend more effectively than complex models. * **The "Curse of Dimensionality" in KNN:** The **KNN Regressor** performed poorly (**R^2: 0.5235**). Since our dataset has many features after One-Hot Encoding, the "distance" between students becomes less meaningful, causing KNN to struggle with finding truly similar neighbors in a high-dimensional space. * **Structural Simplicity:** While the **Decision Tree** performed well (**R^2: 0.8171**), it still fell short of the baseline. This is likely because trees create "axis-aligned" splits (rectangular decision boundaries), which might not fit the smooth, continuous progression of grades as well as a regression line.

1.8.3 3. Model Disagreements and Failures

- **Disagreement on Complexity:** There is a significant performance gap between the Baseline and KNN. This indicates that a simple approach is far better for this specific data structure than a proximity-based one.
- **Common Failure Points:** All models show a **CV Standard Deviation** between **0.04 and 0.05**. This suggests that while the models are generally stable, there is a small amount of variance depending on which students end up in the training vs. validation sets.
- **Residual Errors:** Even our best model has an **MAE of 0.76**. This means that on average, our predictions are off by nearly a full point. This error likely stems from "unobserved variables" (like a student's health on exam day or personal motivation) that are not captured in the CSV file.

1.9 8. Progress log

1.9.1 What the Team Tried:

- **Data Preparation:** We cleaned the `student-por.csv` dataset, handled categorical variables via dummy encoding, and standardized all numerical features using `StandardScaler` to ensure fair comparison between models like KNN and Linear Regression.
- **Modeling Pipeline:** We established a robust evaluation pipeline using a 80/20 train-test split and implemented 5-fold cross-validation for every model to ensure we weren't just "getting lucky" with our test split.

1.9.2 What Worked:

- **Feature Scaling:** This was essential for KNN; without it, features with larger scales (like `absences`) would have dominated the distance calculations.
- **Baseline Selection:** Starting with Linear Regression gave us a very high bar to beat, which helped us realize that for this specific academic dataset, simpler models are highly effective.

1.9.3 Blockers and Challenges:

- **Interpretation of Metrics:** Initially, we only looked at Accuracy, but quickly realized that for a **Regression** task (predicting a continuous grade), we needed to switch to RMSE, MAE, and R^2.
- **Overfitting Concerns:** We noticed that the Decision Tree's R^2 on the training set was much higher than on the test set. To fix this, we limited the `max_depth` to 5, which helped bridge the gap and improved our cross-validation stability.
- **Technical Hurdle:** We faced a minor issue where `pd.get_dummies` created a different number of columns in the training and test sets. We resolved this by aligning the columns after the split to ensure the models wouldn't crash during `predict()`.